Mawlana Bhashani Science and Technology University (MBSTU)

Santosh, Tangail, Bangladesh

[Department]

EduConnect

Smart Student–Tutor Matching System

Final Capstone Project Proposal and Prototyping

Part I

Submitted by: **[Student Name]**
Student ID: **[Student ID]**
Course Title: **[Course Title]**
Course Code: **[Course Code]**
Instructor: **[Instructor Name]**
Submission Date: **[Submission Date]**

Red bracketed items are placeholders to be replaced before submission.

Source repository: `EduConnect` (see `README.md`).

Abstract

Students in Bangladeshi universities often look for private tutors through word of mouth, social-media groups or notice boards. This process is slow and opaque: students cannot easily compare tutors by subject expertise, price, teaching mode and real availability, and tutors lose students because schedules are coordinated by hand. **EduConnect** is a web-based system that matches a student's stated requirements (subject, budget, teaching mode, preferred days and time window, minimum rating) to suitable tutors using a transparent weighted-scoring model, shows an explained compatibility score for each tutor, checks live availability, and lets the student request a booking while the system prevents double-booking and overlapping sessions.

This document is the Part I deliverable. It presents the problem, objectives, scope, requirements, feasibility analysis, the chosen process model (incremental development with evolutionary prototyping), an eight-week schedule, system architecture and six UML/structured-analysis diagrams (DFD, use case, activity, ERD, class and object), a risk analysis, a testing plan and the expected outcomes. A working prototype built with HTML, CSS and vanilla JavaScript demonstrates ten classes (plus one enumeration) that also appear in the class diagram, including the weighted matching algorithm, availability checking, conflict-free booking and booking-status display. The report, diagrams, database schema and prototype were designed together so that names, attributes and rules are consistent across all of them.

Keywords: tutor matching, recommender system, weighted scoring, booking, UML, prototype, MBSTU.

Contents

Abstract i

1 Introduction 1

1.1 Background . . . 1

1.2 Project Overview . . . 1

1.3 Document Organisation . . . 1

2 Problem Statement 1

3 Objectives 2

3.1 General Objective . . . 2

3.2 Specific Objectives . . . 2

4 Scope 2

4.1 In Scope (Part I) . . . 2

4.2 Out of Scope for the Prototype (Planned for the Full System) . . . 2

4.3 Assumptions and Constraints . . . 3

5 Requirements 3

5.1 Stakeholders and User Classes . . . 3

5.2 Functional Requirements . . . 3

5.3 Non-Functional Requirements . . . 4

6 Feasibility Analysis 5

6.1 Technical Feasibility . . . 5

6.2 Economic Feasibility . . . 5

6.3 Operational Feasibility . . . 5

6.4 Schedule Feasibility . . . 5

6.5 Legal, Ethical and Social Feasibility . . . 5

7 Software Process Model 6

7.1 Candidate Models . . . 6

7.2 Chosen Model and Justification . . . 6

8 Project Schedule 7

9 System Design 10

9.1 Architecture . . . 10

9.2 Data Flow Diagram 10
9.3 Use Case Diagram 12
9.4 Activity Diagram 12
9.5 Entity Relationship Diagram and Database Schema 13
9.6 Class Diagram 14
9.7 Object Diagram 15
9.8 Matching Algorithm 16

10 Prototype **17**
10.1 Implemented Features 17
10.2 Class-to-File Mapping 17
10.3 Limitations 18
10.4 How to Run 18

11 Risk Analysis **18**

12 Testing Plan **18**
12.1 Strategy 18
12.2 Representative Test Cases 19

13 Expected Outcomes **19**

14 Conclusion **20**

References **21**

A Requirement Traceability **22**

List of Figures

1 Eight-week Gantt chart with dependencies and milestones . . . 8
2 System architecture (green = implemented in the prototype) . . . 10
3 DFD Level 0 – context diagram . . . 10
4 DFD Level 1 . . . 11
5 Use case diagram . . . 12
6 Activity diagram: match and book a tutor . . . 13
7 Entity relationship diagram (crow's foot notation) . . . 14
8 Class diagram . . . 15
9 Object diagram of the demo scenario . . . 16

List of Tables

1 Specific, measurable objectives . . . 2
2 Functional requirements . . . 4
3 Non-functional requirements . . . 4
4 Illustrative cost estimate (assumptions, not quotations) . . . 5
5 Comparison of candidate process models for this project . . . 6
6 Increments and their mapping to weeks . . . 7
7 Task list with dependencies and deliverables . . . 9
9 Milestones . . . 9
10 Scoring components (default weights sum to 100) . . . 16
11 Worked example: Rahim Uddin (T001) for the demo requirement . . . 17
12 Classes in the class diagram and their implementation . . . 17
13 Risk register . . . 18
14 Representative test cases (full list in the repository checklist) . . . 19
15 Traceability from requirements to design, code and tests . . . 22

1 Introduction

1.1 Background

Private tutoring is a widespread complement to formal education in many countries, including Bangladesh [1]. University students commonly seek help in foundational courses such as calculus, data structures, physics and English composition. At the same time, internet access and smartphone use are high enough among young Bangladeshis that a web-based service is a realistic delivery channel [2].

Today, matching is informal: a student posts a request in a Facebook group, collects replies, compares prices by chat and negotiates a time. Each step is manual, information is incomplete, and nobody checks whether the tutor's free time truly fits the student's timetable.

1.2 Project Overview

EduConnect is a smart student–tutor matching system. The student enters requirements once; the system filters tutors who teach the subject, scores each candidate on six weighted factors, ranks them, explains every score, and allows the student to request a free slot. A booking moves through the states *Pending*, *Confirmed*, *Completed* or *Cancelled*, and the system refuses any request that would create a double booking.

1.3 Document Organisation

Sections 2–6 define the problem, objectives, scope, requirements and feasibility. Section 7 justifies the process model and Section 8 gives the schedule. Section 9 presents the design, Section 10 the prototype, Sections 11–12 the risks and test plan, and Section 13 the expected outcomes.

2 Problem Statement

Students who need academic help face four connected problems:

1. **Discovery:** no single place lists tutors with verified subjects, prices and ratings.
2. **Fit:** choosing a tutor means balancing expertise, budget, mode (online or in person), rating and schedule; students do this mentally and inconsistently.
3. **Availability:** a tutor who looks perfect may have no free slot at the needed time, which is discovered only after several messages.
4. **Coordination:** manual scheduling causes double bookings and overlapping commitments for both students and tutors.

Problem to solve: there is no transparent, availability-aware, conflict-free way for a student to find and book a suitable tutor quickly. EduConnect addresses this gap.

3 Objectives

3.1 General Objective

Design and prototype a web system that recommends and books the most suitable tutor for a student's requirements with explainable results and no scheduling conflicts.

3.2 Specific Objectives

Table 1: Specific, measurable objectives

ID	Objective	Measure of success
O1	Capture structured student requirements	Form validates 6 inputs; invalid input blocked with messages
O2	Rank tutors with a weighted model	Six factors, weights sum to 100%; scores in 0–100
O3	Explain every compatibility score	Per-factor weight, fit and points shown for each tutor
O4	Check tutor availability	Only free slots offered; matching slots highlighted
O5	Prevent booking conflicts	0 double bookings in conflict test cases (TC09–TC12)
O6	Show booking status	Four states displayed with allowed transitions only
O7	Produce consistent design artefacts	Six diagrams whose names match prototype classes
O8	Deliver in eight weeks	All milestones M1–M6 met (Section 8)

4 Scope

4.1 In Scope (Part I)

- Complete proposal, six diagrams, database schema and eight-week plan.
- Browser-based prototype implementing requirement input, weighted matching with explanations, availability checking, booking with conflict prevention and booking-status display using synthetic data.
- Ten JavaScript classes (plus the `BookingStatus` enumeration) matching the class diagram; automated unit tests for the logic.

4.2 Out of Scope for the Prototype (Planned for the Full System)

- User registration, authentication, administrator verification of tutors and subject management (use cases UC1, UC2, UC11, UC13, UC14).

- Server-side database, payments, reviews, chat, push/SMS notifications and mobile apps.
- Calendar dates, time zones and recurring-session management.

4.3 Assumptions and Constraints

- Availability is weekly and recurring (Saturday–Thursday working week; Friday is a holiday).
- Prices are in Bangladeshi taka (BDT) per hour; ratings are on a 0–5 scale.
- One student developer, eight weeks, no paid infrastructure; the stack must run offline in a browser.

5 Requirements

5.1 Stakeholders and User Classes

Student (searches and books), **Tutor** (publishes subjects and availability, answers requests), **Administrator** (verifies tutors, manages subjects), and the **course instructor/evaluator** (assesses the capstone).

5.2 Functional Requirements

Requirements follow the guidance of ISO/IEC/IEEE 29148 [3]. The "Prototype" column shows whether Part I implements the requirement.

Table 2: Functional requirements

ID	Requirement	Use case	Prototype
FR1	Users shall register and log in with a role (student, tutor, admin).	UC1	Planned
FR2	Users shall manage their profile data.	UC2	Planned
FR3	A student shall submit requirements: subject, maximum budget, mode, days, time window, minimum rating.	UC3	Yes
FR4	The system shall rank tutors who teach the subject by a weighted compatibility score (0–100).	UC4	Yes
FR5	The system shall explain each score by factor, weight and points.	UC5	Yes
FR6	The system shall check tutor availability and highlight free slots matching the request.	UC6	Yes
FR7	A student shall request a booking for a free slot.	UC7	Yes
FR8	The system shall reject a booking if the slot is taken or overlaps another active booking of the student or tutor.	UC8	Yes
FR9	The system shall display booking status (Pending, Confirmed, Completed, Cancelled).	UC9	Yes
FR10	A student shall cancel a pending or confirmed booking, releasing the slot.	UC10	Yes
FR11	A tutor shall manage availability slots.	UC11	Seed data
FR12	A tutor shall accept, decline or complete a booking.	UC12	Simulated
FR13	An administrator shall verify tutor profiles.	UC13	Planned
FR14	An administrator shall manage the subject catalogue.	UC14	Planned

5.3 Non-Functional Requirements

Quality attributes are drawn from ISO/IEC 25010 [4].

Table 3: Non-functional requirements

ID	Attribute	Requirement
NFR1	Usability	Responsive from 360 px phones to desktops; keyboard-operable controls; target WCAG 2.1 AA contrast and labels [5]; follows common usability heuristics [6].
NFR2	Performance	Ranking of up to 1 000 tutors in under one second on a typical laptop (to be measured in system testing).
NFR3	Reliability / integrity	No double booking: enforced in application logic and, in the full system, by a database partial unique index.
NFR4	Security	Hashed passwords, HTTPS, server-side validation and role-based access (full system).
NFR5	Maintainability	Modular classes, one responsibility per file, automated unit tests for core logic.
NFR6	Portability	Runs in current Chrome, Firefox, Edge and Safari without build tools.
NFR7	Explainability	Every ranking decision is decomposable into factor-level points shown to the user.

6 Feasibility Analysis

6.1 Technical Feasibility

The prototype uses only standard web technologies (HTML5, CSS3, ECMAScript classes [7]). The weighted-sum model is a well-established multi-criteria decision method [8] and a common baseline in recommender systems [9]. All algorithms are $O(n)$ per request over n candidate tutors, so a browser handles realistic data volumes. The full system can use any open-source relational database and web framework. **Verdict: feasible.**

6.2 Economic Feasibility

Table 4: Illustrative cost estimate (assumptions, not quotations)

Item	Estimate	Note
Development tools (VS Code, Git, Node.js, PlantUML, TeX)	0 BDT	Free and open source
Prototype hosting (GitHub Pages)	0 BDT	Static site
Internet data for 8 weeks	[amount] BDT	Student's existing plan
Developer effort	≈120 hours	Student time, not charged
Printing and binding of report	[amount] BDT	If a hard copy is required
Full-system VPS (future)	≈ [amount] BDT/month	Low-tier virtual server

Since the prototype needs no paid resources, Part I is economically feasible. A future deployment could be funded by a small booking commission or sponsorship; this is outside Part I.

6.3 Operational Feasibility

Students already use web forms and messaging apps, so the interaction (fill a form, read a ranked list, click a slot) is familiar. Tutors need only to publish subjects and weekly availability. Because rankings are explained, users can judge the recommendation rather than trust a black box.

6.4 Schedule Feasibility

The scope is deliberately limited to a prototype with synthetic data. Section 8 breaks the work into 13 tasks with finish-to-start dependencies, which fit within eight weeks with one week of final review.

6.5 Legal, Ethical and Social Feasibility

The prototype stores no real personal data. A production system must comply with applicable Bangladeshi data-protection and ICT law, obtain consent for personal data, avoid discriminatory ranking (for example, new tutors without ratings must not be permanently hidden), and keep tutor verification transparent.

7 Software Process Model

7.1 Candidate Models

Table 5: Comparison of candidate process models for this project

Model	Strengths	Weaknesses here	Fit
Waterfall [10]	Clear phases, easy documentation	Late feedback; changes are costly; prototype arrives last	Low
Spiral [11]	Strong risk management	Heavy overhead for one developer and 8 weeks	Medium
Scrum / Agile [12, 13]	Short iterations, frequent feedback	Full ceremonies are too heavy for one person	Medium
Incremental + evolutionary prototyping [14]	Working increments, early demos, lightweight	Needs discipline in documentation	**High**

7.2 Chosen Model and Justification

EduConnect uses the **incremental model with evolutionary prototyping**, organised in one-week sprints borrowed from Scrum (a short plan, a working increment, a short review). The reasons are:

1. **Deliverable is a prototype:** the assignment explicitly requires a working prototype, so a model that produces runnable increments early is natural.
2. **Time-boxed:** eight fixed weeks suit fixed-length sprints with a milestone at the end of each increment.
3. **Moderately stable but refinable requirements:** the core features are known, but weights, input fields and the explanation format benefit from feedback after each increment.
4. **Risk reduction:** the riskiest parts (matching quality and conflict-free booking) are built and tested first (increment 3), not at the end.
5. **Small team:** one developer cannot sustain full Scrum roles, but can run the lightweight cycle of plan, build, test and review.

Table 6: Increments and their mapping to weeks

Increment	Weeks	Content	Output
1	1–2	Requirements, literature, SRS, use cases, ERD	Baseline (M1)
2	3	DFD, activity, class and object models; wireframes	Design complete (M2)
3	4–5	Matching algorithm; availability and booking logic	Tested core logic (M3)
4	6–7	UI integration, status display, testing	Integrated prototype (M4), tested (M5)
5	8	Final review, packaging, submission	Submission (M6)

8 Project Schedule

The plan below is generated from `docs/schedule/tasks.csv` so that the chart and tables stay consistent. All dependencies are finish-to-start, and every task starts in a later week than its predecessors end.

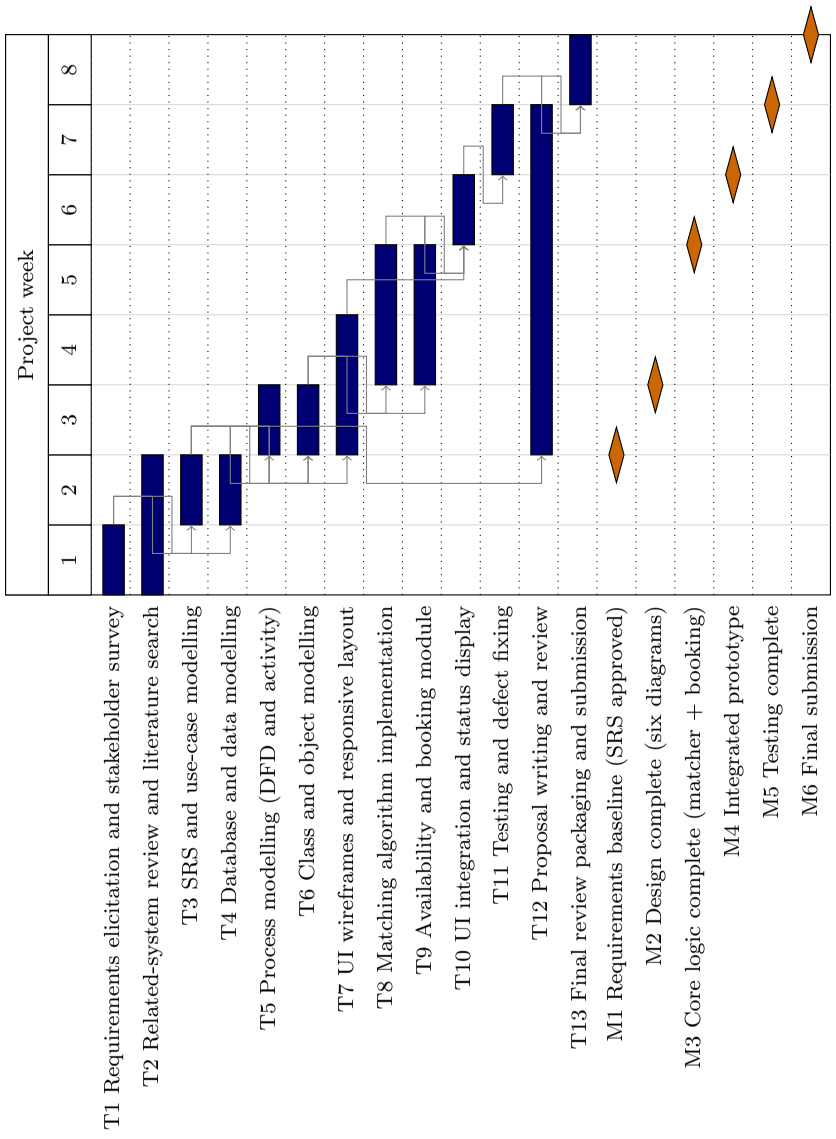


Figure 1: Eight-week Gantt chart with dependencies and milestones

Table 7: Task list with dependencies and deliverables

ID	Task	Weeks	Depends on	Deliverable
T1	Requirements elicitation and stakeholder survey	1	–	Requirement notes; survey summary
T2	Related-system review and literature search	1–2	–	Literature summary; reference list
T3	SRS and use-case modelling	2	T1	SRS draft; Use Case Diagram
T4	Database and data modelling	2	T1	ERD; relational schema
T5	Process modelling (DFD and activity)	3	T3	DFD (L0/L1); Activity Diagram
T6	Class and object modelling	3	T3,T4	Class Diagram; Object Diagram
T7	UI wireframes and responsive layout	3–4	T3	Wireframes; CSS foundation
T8	Matching algorithm implementation	4–5	T6	matcher.js; unit tests
T9	Availability and booking module	4–5	T6	bookingManager.js; conflict tests
T10	UI integration and status display	6	T7,T8,T9	Integrated prototype
T11	Testing and defect fixing	7	T10	Test results; testing checklist
T12	Proposal writing and review	3–7	T3	main.tex; proposal PDF
T13	Final review packaging and submission	8	T11,T12	GitHub repository; ZIP

Table 9: Milestones

Milestone	Week	Gate task	Exit criterion
M1 Requirements baseline (SRS approved)	2	T3,T4	SRS signed off
M2 Design complete (six diagrams)	3	T5,T6	All six diagrams reviewed
M3 Core logic complete (matcher + booking)	5	T8,T9	Unit tests pass
M4 Integrated prototype	6	T10	Demo scenario runs end to end
M5 Testing complete	7	T11	Checklist signed off
M6 Final submission	8	T13	Submitted package

9 System Design

9.1 Architecture

The full system follows a three-tier architecture (Figure 2): a responsive browser UI, an application tier with matching, booking, availability, authentication and notification services, and a relational database. The Part I prototype implements the highlighted components in the browser and stores bookings in `localStorage`.

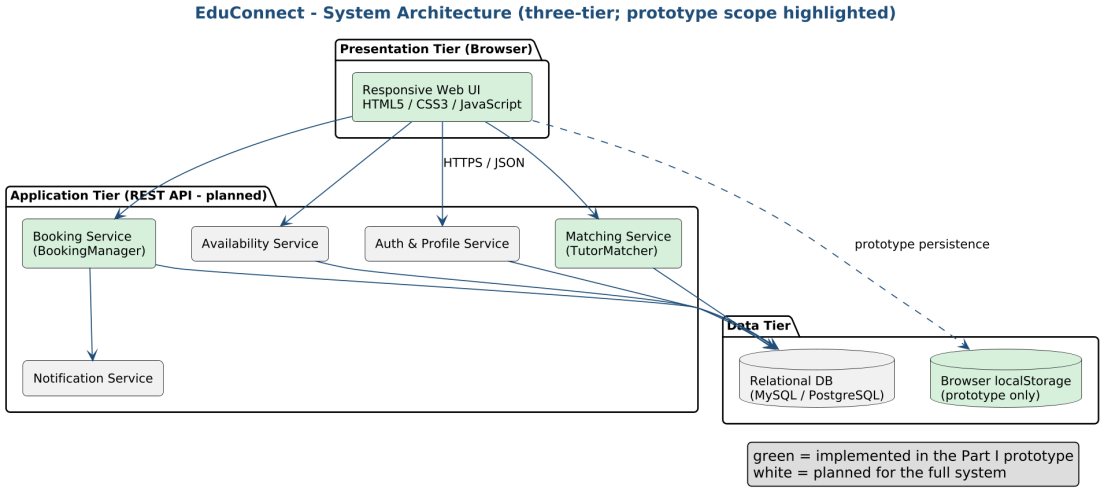


Figure 2: System architecture (green = implemented in the prototype)

9.2 Data Flow Diagram

Figure 3 shows the context (Level 0) and Figure 4 the Level 1 decomposition into six processes (accounts, requirements, matching, availability, booking and status update) and five data stores. Data store D3 is shared by matching, availability and booking, which is where conflicts are detected.

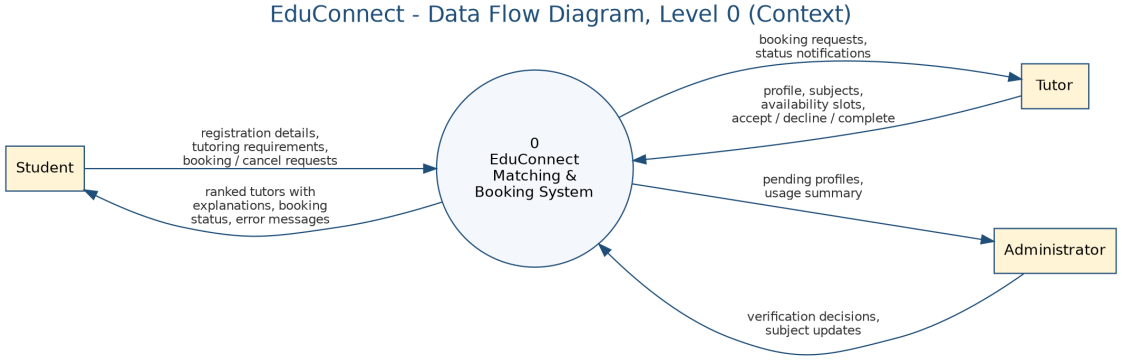


Figure 3: DFD Level 0 – context diagram

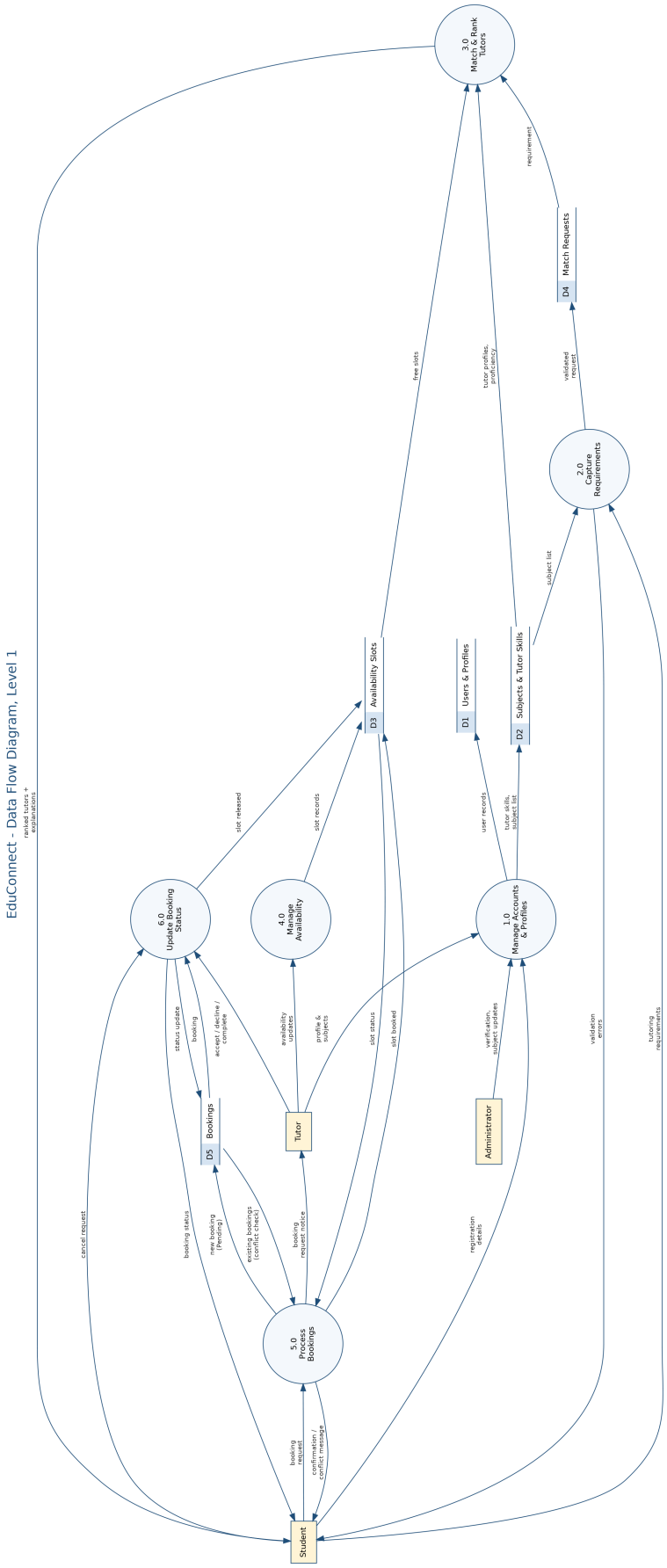


Figure 4: DFD Level 1

9.3 Use Case Diagram

Fourteen use cases are defined for three actors (Figure 5). *View Ranked Tutor Matches* includes submitting requirements, explaining scores and checking availability; *Book Session* includes conflict prevention.

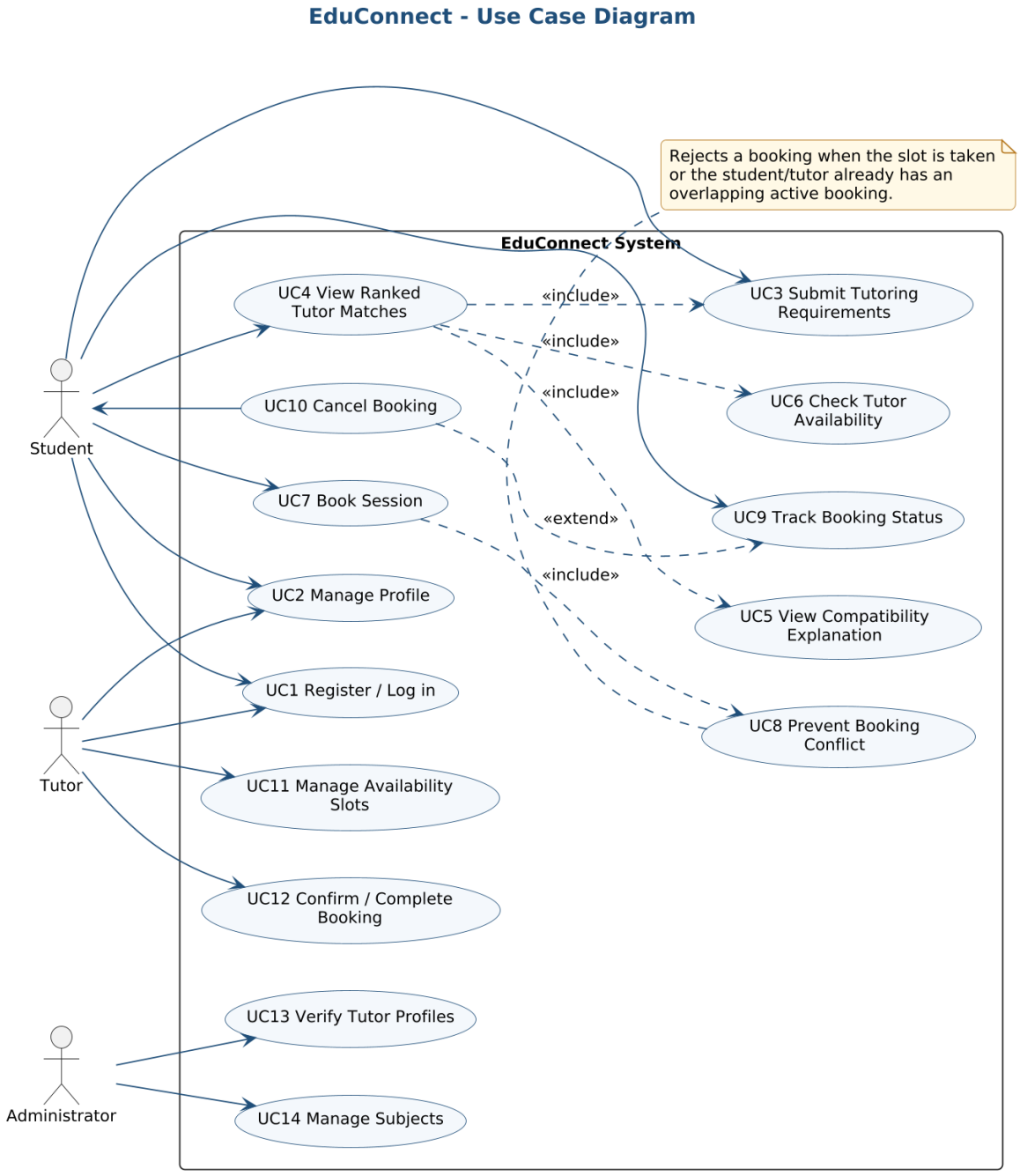


Figure 5: Use case diagram

9.4 Activity Diagram

Figure 6 models the main scenario with three swim-lanes (Student, System, Tutor), including input validation, ranking, the conflict check and the tutor's response.

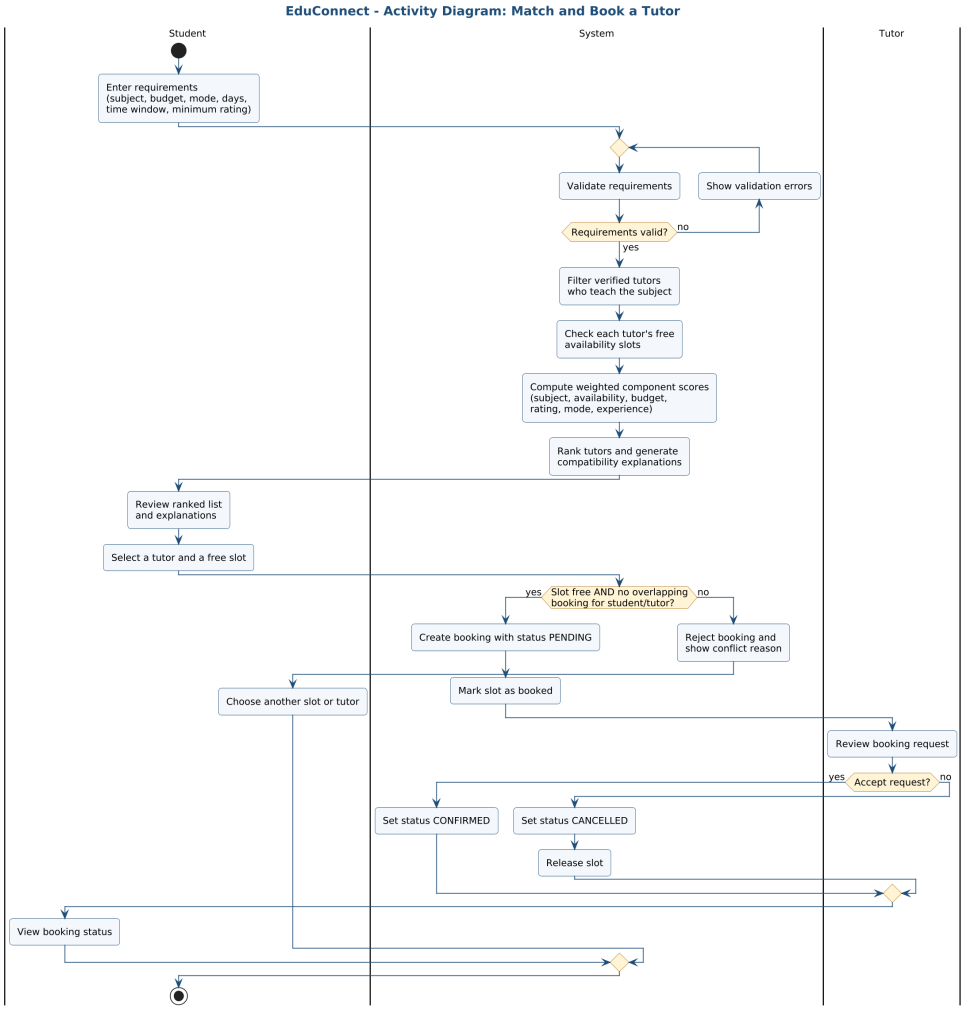


Figure 6: Activity diagram: match and book a tutor

9.5 Entity Relationship Diagram and Database Schema

The ERD (Figure 7) has eight entities. Users have a role and an optional student or tutor profile; tutors teach subjects with a proficiency (1–5) through the associative entity `tutor_subjects`; tutors offer availability slots; students submit match requests and make bookings. The business rule "at most one active booking per slot" is enforced by a partial unique index in `database/schema.sql`, which was tested with SQLite (`tests/test_schema.py`). The design follows standard relational modelling practice [15].

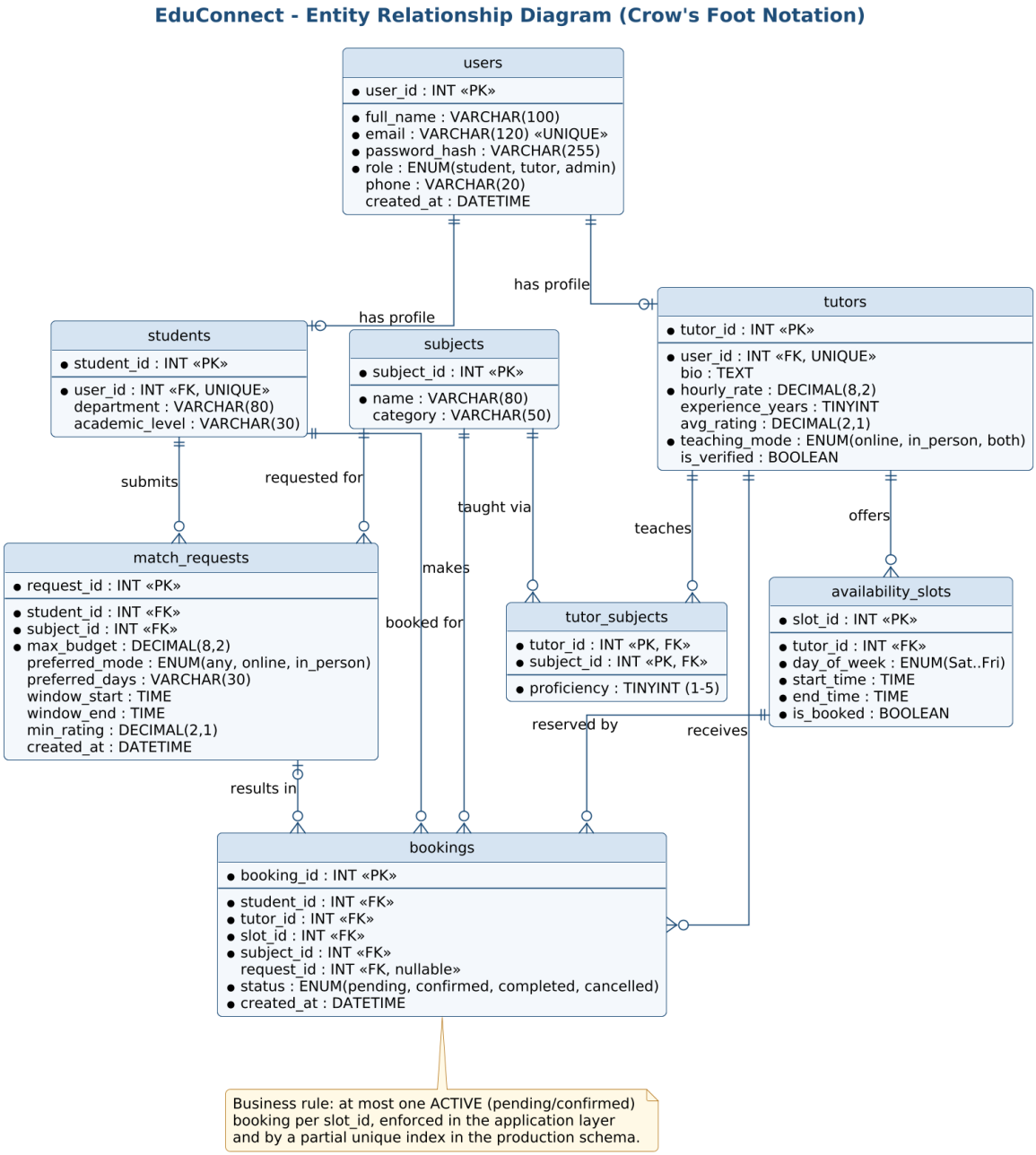


Figure 7: Entity relationship diagram (crow's foot notation)

9.6 Class Diagram

The class diagram (Figure 8) contains the ten classes and one enumeration implemented in the prototype. `Student` and `Tutor` specialise `User`; `Tutor` composes `AvailabilitySlot`; `TutorMatcher` produces `MatchResult`; `BookingManager` aggregates `Booking`. Notation follows UML [16].

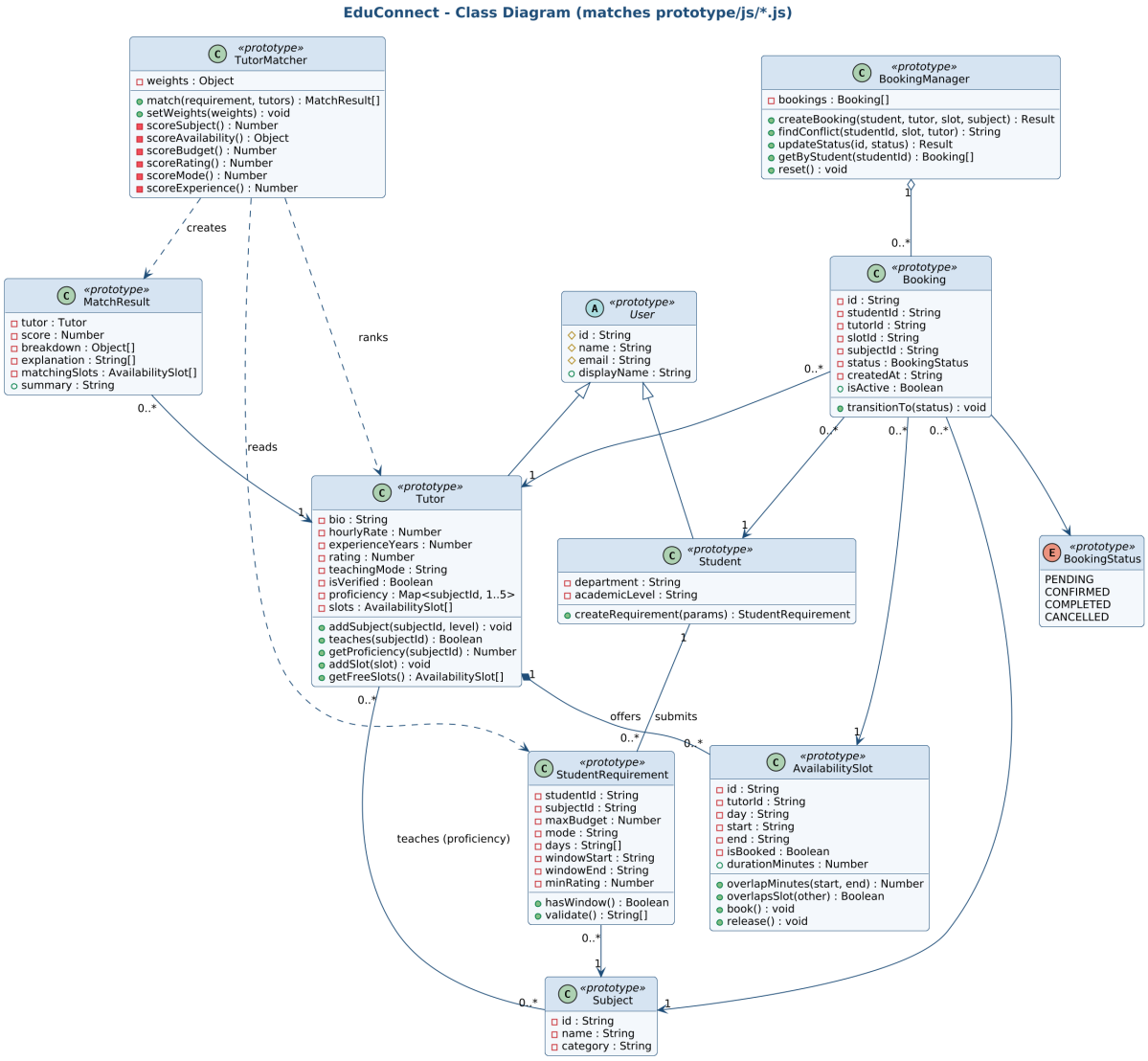


Figure 8: Class diagram

9.7 Object Diagram

Figure 9 is a snapshot of the demo scenario: student Nusrat Jahan asks for Data Structures online, at most 650 BDT/h, on Saturday or Monday between 18:00 and 20:00; the matcher returns Rahim Uddin first with a score of 93.2, and booking B002 is created in the *Pending* state.

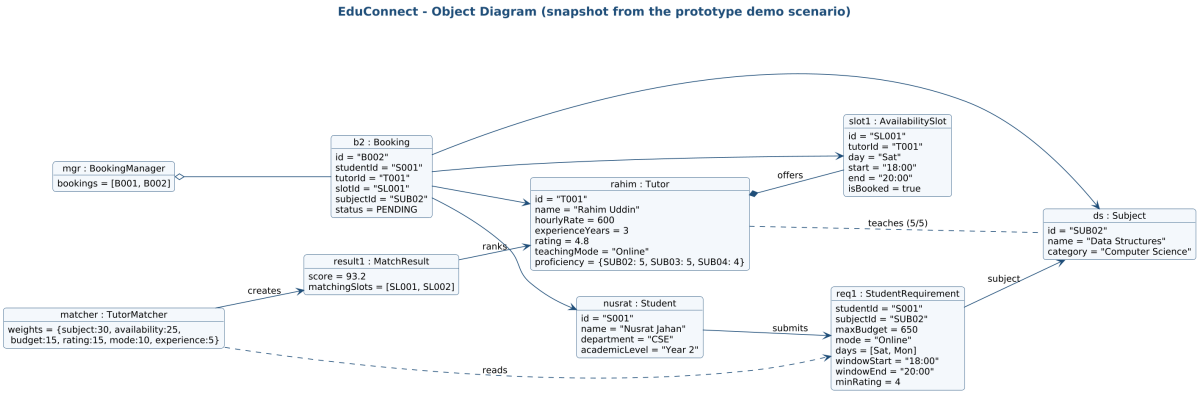


Figure 9: Object diagram of the demo scenario

9.8 Matching Algorithm

A tutor is a candidate only if verified and teaching the requested subject. For each candidate, six component scores $c_i \in [0, 1]$ are combined with weights w_i [8]:

$$\text{Score} = 100 \cdot \frac{\sum_{i=1}^{6} w_i \, c_i}{\sum_{i=1}^{6} w_i}$$

Table 10: Scoring components (default weights sum to 100)

Component	**Weight**	**Component score c_i**
Subject expertise	30	proficiency / 5
Availability fit	25	With a time window: best overlap of a free slot (on requested days) divided by window length. Without: $\min(1, \text{free slots}/3)$.
Budget fit	15	Rate $\leq$ budget: $1 - 0.3 \cdot$rate/budget (0.7–1.0). Over budget: $\max(0,\ 0.7 - (\text{rate} - \text{budget})/\text{budget})$.
Rating	15	rating / 5, halved if below the student's minimum rating
Teaching mode	10	1 if student has no preference, tutor teaches both, or modes equal; else 0
Experience	5	$\min(\text{years}/5,\ 1)$

Ties are broken by rating (higher first) and then rate (lower first). Table 11 shows the worked example from the object diagram, reproduced by the prototype and its unit tests.

Table 11: Worked example: Rahim Uddin (T001) for the demo requirement

Factor	Weight	Fit	Points
Subject expertise (5/5)	30%	100%	30.0
Availability (2 slots cover window)	25%	100%	25.0
Budget (600 vs 650 BDT/h)	15%	72%	10.8
Rating (4.8/5)	15%	96%	14.4
Mode (Online = Online)	10%	100%	10.0
Experience (3 years)	5%	60%	3.0
Total	100%		**93.2**

10 Prototype

10.1 Implemented Features

The prototype (`prototype/index.html`) is a responsive single-page interface that runs from a local file or any static host with no build step.

- **Requirement input** with validation (FR3).
- **Weighted ranking** with adjustable priorities, a score bar and a "Why this score?" table (FR4, FR5).
- **Availability checking:** free slots highlighted when they fit the requested days and time window; booked slots are disabled (FR6).
- **Booking and conflict prevention:** slot already taken, tutor overlap and student overlap are rejected with an explanation (FR7, FR8).
- **Booking status display** with simulated tutor actions and student cancellation (FR9, FR10, FR12).

10.2 Class-to-File Mapping

Table 12: Classes in the class diagram and their implementation

Class	File	Responsibility
User, Student, Tutor	`js/models.js`	People; tutor holds proficiency map and slots
Subject	`js/models.js`	Course catalogue entry
AvailabilitySlot	`js/models.js`	Weekly slot, overlap maths, book/release
StudentRequirement	`js/models.js`	Requirement data and validation
MatchResult, TutorMatcher	`js/matcher.js`	Scoring, ranking, explanations
Booking, BookingManager	`js/bookingManager.js`	Status lifecycle, conflict detection, persistence

10.3 Limitations

The prototype has no server, authentication or real database; data are synthetic; slots are recurring weekly without dates; tutor responses are simulated; weights are hand-chosen and not learned; and client-side conflict checks must be repeated server-side in a production system. These are stated in the user interface and in the README.

10.4 How to Run

Open `prototype/index.html` in a browser, or run `python3 -m http.server` in the `prototype` folder. Run `node tests/run-tests.js` for the logic tests.

11 Risk Analysis

Probability (P) and impact (I) are rated Low (L), Medium (M) or High (H).

Table 13: Risk register

ID	Risk	P	I	Mitigation / contingency
R1	Requirement creep during prototyping	M	H	Freeze scope at M1; log new ideas as "future work"
R2	Schedule slip (illness, exams, load-shedding)	M	H	One-week buffer in week 8; work offline; Git commits daily
R3	Poor or biased ranking (e.g. new tutors disadvantaged)	M	M	Transparent scores; adjustable weights; neutral rating default planned for tutors without reviews
R4	Double booking in production due to concurrent requests	M	H	Database partial unique index and transactions (schema tested)
R5	Cold start: too few tutors or ratings	H	M	Seed data in prototype; admin-assisted onboarding in full system
R6	Privacy or misuse of personal data	M	H	Collect minimum data; hash passwords; consent and retention policy
R7	Browser storage unavailable or corrupted	L	L	Prototype falls back to in-memory seed data
R8	Loss of work or file corruption	L	H	Git repository, remote backup, ZIP snapshot at each milestone
R9	Inconsistency between report, diagrams and code	M	M	Single class list; consistency checks listed in the test plan

12 Testing Plan

12.1 Strategy

Testing follows the usual levels [17]: unit tests for model, matcher and booking logic (`tests/run-tests.js`); a schema test (`tests/test_schema.py`); integration and system testing through the manual checklist (`docs/TESTING_CHECKLIST.md`); usability review against heuristics

[6]; and acceptance testing against requirements FR3–FR10. Exit criteria: all automated tests pass, no open high-severity defect, and every checklist item for FR3–FR10 passes.

12.2 Representative Test Cases

Table 14: Representative test cases (full list in the repository checklist)

ID	Req.	Test	Expected result
TC01	FR3	Submit empty subject or negative budget	Inline errors; no search
TC02	FR3	Window end earlier than start	Validation error
TC03	FR4	Demo scenario (Data Structures)	Only T001, T002, T006 listed; T001 first
TC04	FR4	Scores sorted, within 0–100	Descending order; all in range
TC05	FR5	Open "Why this score?"	Six factors; points sum to total
TC06	FR6	Tutor with no free slot on chosen days	Availability fit 0%
TC07	FR4	Change weights (e.g. budget only)	Ranking changes; all-zero weights rejected
TC08	FR7	Book a free slot	Booking Pending; slot disabled
TC09	FR8	Second student books same slot	Rejected: slot taken
TC10	FR8	Same student books overlapping slots with two tutors	Rejected: overlap
TC11	FR8	Tutor not teaching the subject	Rejected
TC12	FR8	Booking persists after page reload	Still listed; slot still blocked
TC13	FR9	Pending → Confirmed → Completed	Statuses update; illegal jumps rejected
TC14	FR10	Cancel booking	Status Cancelled; slot re-bookable
TC15	NFR1	View on 360 px width	No horizontal page scroll; controls usable
TC16	NFR3	Insert two active bookings for one slot in SQL	Second insert rejected

13 Expected Outcomes

1. A reviewed proposal that defines problem, scope, requirements, feasibility, process, schedule, risks and tests.
2. Six consistent diagrams plus architecture and DFD Level 0, each with editable source (PlantUML and Graphviz).
3. A relational schema consistent with the ERD and verified for double-booking prevention.
4. A working prototype demonstrating explainable ranking and conflict-free booking, with automated tests (19 logic tests and 5 schema checks).
5. A professional GitHub repository with README, setup instructions and a ZIP archive for submission.
6. A foundation for the full system: authentication, server-side persistence, notifications, reviews and learned weights.

14 Conclusion

EduConnect turns an informal, error-prone search into a structured and explainable process. The proposal shows that the idea is feasible within eight weeks, that the design artefacts are mutually consistent, and that the prototype already demonstrates the most valuable behaviour: ranking tutors fairly and refusing conflicting bookings.

References

[1] M. Bray, "Confronting the shadow education system: What government policies for what private tutoring?," tech. rep., UNESCO International Institute for Educational Planning, Paris, France, 2009.

[2] S. Kemp, "Digital 2024: Bangladesh." DataReportal, https://datareportal.com/reports/digital-2024-bangladesh, 2024. Accessed: [access date].

[3] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 Systems and software engineering – Life cycle processes – Requirements engineering*, 2018.

[4] ISO/IEC, *ISO/IEC 25010:2011 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*, 2011.

[5] World Wide Web Consortium (W3C), "Web content accessibility guidelines (WCAG) 2.1." https://www.w3.org/TR/WCAG21/, 2018. Accessed: [access date].

[6] J. Nielsen, "10 usability heuristics for user interface design." Nielsen Norman Group, https://www.nngroup.com/articles/ten-usability-heuristics/, 1994. Accessed: [access date].

[7] Ecma International, "ECMA-262: ECMAScript language specification." https://tc39.es/ecma262/. Accessed: [access date].

[8] E. Triantaphyllou, *Multi-Criteria Decision Making Methods: A Comparative Study*. Dordrecht, The Netherlands: Kluwer Academic Publishers, 2000.

[9] F. Ricci, L. Rokach, and B. Shapira, eds., *Recommender Systems Handbook*. New York, NY: Springer, 3 ed., 2022.

[10] I. Sommerville, *Software Engineering*. Harlow, UK: Pearson, 10 ed., 2016.

[11] B. W. Boehm, "A spiral model of software development and enhancement," *Computer*, vol. 21, no. 5, pp. 61–72, 1988.

[12] K. Beck *et al.*, "Manifesto for agile software development." https://agilemanifesto.org/, 2001. Accessed: [access date].

[13] K. Schwaber and J. Sutherland, "The Scrum Guide." https://scrumguides.org/, 2020. Accessed: [access date].

[14] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*. New York, NY: McGraw-Hill Education, 9 ed., 2020.

[15] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*. Hoboken, NJ: Pearson, 7 ed., 2016.

[16] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Boston, MA: Addison-Wesley, 3 ed., 2003.

[17] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*. Hoboken, NJ: John Wiley & Sons, 3 ed., 2011.

A Requirement Traceability

Table 15: Traceability from requirements to design, code and tests

Req.	Diagram element	Code	Tests
FR3	UC3; `StudentRequirement`; `match_requests`	`models.js`, `app.js`	TC01, TC02
FR4, FR5	UC4, UC5; `TutorMatcher`, `MatchResult`	`matcher.js`	TC03–TC05, TC07
FR6	UC6; `AvailabilitySlot`	`models.js`, `matcher.js`	TC06
FR7, FR8	UC7, UC8; `BookingManager`; `bookings`	`bookingManager.js`	TC08–TC12, TC16
FR9, FR10	UC9, UC10; `BookingStatus`	`bookingManager.js`, `app.js`	TC13, TC14
NFR1	–	`styles.css`	TC15